□み上げ台本(日本語) — WireGuard for Apache NuttX (Community Over Code, Glasgow)

□番は英語で話す前提。この日□語版は英語版と**1**対**1**で対応する、リハーサル・理解確認用の台□です。 このファイルは `export_ja_script.py` が作ります(構成・時間・合計は英語版と同じ計算、日□語の文は `ja_notes.py`)。手で直さず、英語のノート → `ja_notes.py` の順に直して、`export_en_script.py` と `export_ja_script.py` を実行してください。スライドごとの段落の数が英語版と違うと、生成が止まります。

2026-10-13: 英語版を読み上げ用に作り直したのに合わせて、日□語版も作り直した(3区分、話速 125 語/分、短い文、 アプリ版とデモの扱いの整理)。日□語は英語の語数に合わせたものではないので、時間は英語版の値を使う。

この台本の□み方

各スライドには最大3つの部分があります。

- 話す **— Essential(必ず話す):** 上から順に読む。それだけで、スライド間のつなぎも含めて、完結した発表になる。ゆっくり、はっきり話す前提。
- 話す **— Optional(時間があれば):** 時間があれば足す。飛ばしても、次の文は成り立つ。
- **Presenter Notes(□み上げない):** 出典、デモの手順、確認事項、注意。

時間(英語版と同じ)

話速 125 語/分(英語の語数で計算)、目次のスライドは各10秒、ライブデモは 3:00(話すことと操作)。スライドの切り替えと短い間は、この数字に含む。

	語数(英語)	時間
Essential のみ(目次の間とライブデモを含む)	2770	**24:20**
Essential + Optional	3448	29:46
うちライブデモ	255	3:00
30分枠の余り(Essential のみ)		5:40
30分枠の余り(Essential + Optional)		0:14

部	スライド	Essential	+ Optional
最初: 表紙・自己紹介・目次	1–3	1:32	1:57
① 概要と背景	4–9	4:29	5:24
② 課題と解決	10–14	3:24	3:49
③ 開発計画	15–17	1:45	2:17
④ Version 1: アプリケーション版	18–23	3:28	4:30
⑤ Version 2: カーネルドライバ	24–28	3:17	4:52

部	スライド	**Essential**	**+ Optional**
⑥ ライブデモ	29–31	3:10	3:35
⑦ まとめと振り返り	32–36	3:15	3:22

各スライドの見出しの時間は、Essential の累積(「このあたりにいるはず」)。

削ってはいけないもの

一番大事なのはスライド **28(Testing: What Ran Where)**: 査読者が確認してくる主張が集中している。注記は削らない: FLAT は実機、 KERNEL と PROTECTED は QEMU のみ、KERNEL と PROTECTED の実機はまだ。直後にあった「ping が見逃すもの」と「時計を信じるか」は付録(A1・A2)で、Thank You の後ろ。上の時間には含めず、質問で必要になったときだけ、最後のスライドから下矢印で開く。

前半の物語は削らない(スライド2と4〜14: About Me ′ Overview から Architecture まで): なぜ始めたか、Sony と NuttX、私自身の経験、課題、 なぜ機器に VPN か、なぜ WireGuard か。後半すべての前提になる。スライド**34(What I learned)**も残す — 発表者自身のメッセージ。

時間が足りないときに削る順番**:**

1. Optional をすべて。
2. スライド23(Version 1 の動画の静止画。ライブデモが同じ種類のものを見せる)。
3. スライド22(バグ③ ′ 50ms ポーリング。1行の教訓は一覧表にすでにある)。

ライブデモが止まったら、そう言って、次のスライド(バックアップの動画。時間に含めない)を開くか、まとめへ進む。目次に戻るスライド **(4 · 10 · 15 · 18 · 24 · 29 · 32)**は各**10**秒の一呼吸: ハイライトされた行を指して、一文だけ言う。□当に時間がなければ飛ばしてよい。

---

1 — タイトル (0:00 → 0:27)

画面**:**「 WireGuard for Apache NuttX」を大きく。その下にサブタイトル「Bringing Secure Remote Access to Embedded Devices」。上に Community Over Code Glasgow 2026、下に Satoru Akita / Sony Semiconductor Solutions。右下に小さな QR コード (「 Slides」、最後のスライドと同じ URL)。技術的な情報(wg0 ′ QEMU、ビルドモード、検証環境、バージョン)は載せない。

話す **— Essential(**必ず話す**):**

- 皆さん、こんにちは。日□の Sony Semiconductor Solutions から来ました、Satoru Akita です。
- 今日は、軽量な VPN の WireGuard を Apache NuttX に持ち込んだ経験を話します。
- 目的はシンプルでした: 組み込み機器への、安全な遠隔アクセス。
- なぜこのプロジェクトを始めたか、どう作ったか、直面した難しさ、そして学んだことを話します。

2 — 自己紹介 (0:27 → 1:10)

画面: 見出し「About Me」。上: □人写真・名前の1行・SNS の狐アイコン。下の左: 日□地図 (① 名古屋 生まれ → ② 仙台 大学 → ③ 川崎 現在)と、その下に「Japan」のラベル。 下の右:「 My background & activities」の大きな写真5枚(Rocket / Aerospace Engineering: モデルロケット / DNA Nanorobotics: cryo-TEM 像 / Semiconductor Research: ウェハ上の試作チップ / Robots: ハッカソンのヒューマノイド / Open-Source Conference Volunteer: 運営参加時の写真)。 活動を説明する文章はない。経歴の流れ(分子ロボティクス → … → NuttX)は口頭だけ。

話す — **Essential(必ず話す):**

- まず、簡単に自己紹介をします。
- 私は Sony Semiconductor Solutions の edge AI エンジニアで、application 側から Apache NuttX を使っています。名古屋生まれ、仙台で学び、今は川崎に住んでいます。
- 経歴は幅広いです。大学では機械・航空宇宙工学を学び、ロケットにも取り組みました。学部の研究は DNA ナノロボティクス、修士の研究は半導体でした。ロボットも作りますし、オープンソースのカンファレンスにもボランティアとして参加しています。
- このプロジェクトは、その道のりの最新の到達点です。プロジェクトをお見せします。

話す — **Optional(時間があれば):**

- カンファレンスは Open Source Summit Japan と KubeCon です。ロボットはハッカソンで作っています。
- ここに至る道は、分子ロボット、コンピュータビジョン、edge AI カメラ、そして SPRESENSE 上の組み込みシステムを通っています。
- SNS では、この写真の狐が私です。オープンソースの□も書きました。

3 — 目次 (1:10 → 1:32)

画面: 7つの部。見出しは短い名詞句、その下に「これから何を話すか」を予告する短い1行、右に所要時間 — Overview & background: Project origins · NuttX & Sony · my experience(~5分)/ The Problem & Solution: Remote access challenge · why WireGuard(~4分)/ The Development Plan: Reuse wireguard-lwip · adapt to NuttX(~2分)/ Version 1: Application-Based Implementation: First working tunnel · debugging challenges(~4分)/ Version 2: The Kernel Driver: Why a driver · integration challenges · validation(~5分)/ Live Demo: Secure remote access · Stack-chan control(~3分)/ Summary & Reflections: Achievements · personal reflections · community(~4分)。

話す — **Essential(必ず話す):**

- 全体の流れです。7つの部があります。
- まず背景: なぜ始めたか。次に課題と解決、そして開発計画。その後 Version 1 と Version 2。そしてライブデモと、短いまとめです。
- 約30分で終えます。

4 — 目次 ①Overview & background (1:32 → 1:42)

画面: 目次と同じリスト。①Overview & background だけハイライト、他は薄い。

話す(一言): では、背景から始めます。

間: ハイライトされた行を指して一拍。

5 — プロジェクト概要 (1:42 → 2:14)

画面: `wg0` netdev の図(変更なし)— アプリケーション/NSH ・ NuttX ネットワークスタック、 `eth0`/`wlan0` の隣に `wg0`、その下に UDP ソケット、インターネット、ピア。図の下にカード3枚: Lightweight VPN / Virtual interface / Secure communication。最下段に一文。

話す — **Essential(必ず話す):**

- プロジェクト全体を1枚の絵で見せます。
- 軽量な VPN の WireGuard を Apache NuttX に持ち込みました。wg0 という仮想ネットワークインターフェースとして、普通のインターフェースの隣に現れます。wg0 に送ったパケットは暗号化され、UDP ソケットを通って、普通の WireGuard ピアなら何にでも届きます。これで NuttX のマイコンが、安全に IP パケットをやり取りできます。
- まず、このプロジェクトがどう始まったかを話します。

6 — 経緯: 2月の思いつきから10月の本番まで (2:14 → 3:14)

画面: 2月〜10月の月の横軸。上に出来事 — 2月中旬 GSoC のテーマ一覧を見て調べ始める / 3月上旬 計画を提案 / 3月20日 Alin さんからこのカンファレンスを聞き、CfP を提出 / 4月上旬 GSoC に応募 / 5月上旬 CfP は採択、GSoC は不採択 — それでも続ける / 10月 グラスゴー。下に、矢印の形をした3□の帯 — 5〜7月 NuttX と WireGuard を調べ、作り方を探る → 8月 Version 1(アプリ・FLAT・実機検証)→ 9月〜 Version 2(カーネルドライバ)。

話す — **Essential(必ず話す):**

- 経緯です。
- 2月中旬に、GSoC のテーマ一覧を見て、調べ始めました。3月に計画を提案しました。3月20日に、NuttX コミュニティの Alin さんがこのカンファレンスのことを教えてくれたので、CfP、つまり発表の募集に応募しました。4月の初めに GSoC に応募しました。この2つは別々の応募です。
- 5月の初めに、2つの返事が来ました。CfP は採択、でも GSoC は不採択でした。それでも続けることにしました。
- 5月から7月は、NuttX と WireGuard を調べました。8月は実機での検証を中心に、Version 1 を完成させました。その後、Version 2、カーネルドライバを作りました。
- つまりこの取り組みは GSoC から始まり、一歩ずつ育ちました。まず、NuttX とは何でしょうか。

7 — A Quick Overview of Apache NuttX: NuttX とは何か、どこで使われているか (3:14 → 4:01)

画面: 左: NuttX のロゴと4つの要点(POSIX-like な RTOS / 小さい / 幅広いハードウェア対応 — 15+ アーキテクチャ・300+ ボード / Apache のオープンソース)、標準 API の小さな表 (FreeRTOS: 独自のタスク API / Zephyr: 独自 API とオプションの POSIX 層 / NuttX: POSIX-like が設計方針)。右: 実機の写真タイル — Sony ・ Fitbit(Charge 3 / 4) ・ PX4 のフライトコントローラ、月面ロボット SORA-Q ・ Xiaomi OpenVela(1,000+ 製品)。最下段:「Apache NuttX is a POSIX-like RTOS used in a wide range of real-world embedded devices.」

話す — **Essential(必ず話す):**

- NuttX は、小さなデバイス向けのリアルタイム OS です。POSIX に従うので、Unix のプログラマにはなじみがあります。多くの CPU アーキテクチャとボードに対応していて、Apache のオープンソースプロジェクトです。
- FreeRTOS や Zephyr も良い RTOS です。このプロジェクトで大事なのは、POSIX 的な API です。pthread、ソケット、ファイルがあるので、VPN を普通のネットワークデバイスにできます。
- NuttX は、実際の商用製品でも使われています。画面に例があります：音声製品、フィットネストラッカー、ドローンのコントローラ、そして月面ロボットまで。
- Sony には、NuttX との長い歴史があります。見てみましょう。

話す **— Optional(**時間があれば**):**

- Sony は音声製品と SPRESENSE で NuttX を使っています。Fitbit は Charge 3 や Charge 4 などのフィットネストラッカーで使いました。PX4 のドローン用フライトコントローラも NuttX で動きます。Xiaomi の OpenVela は 1,000 を超える製品で動いていると報告されています。そして小さな月面ロボット SORA-Q が、日□の 2024 年の月面ミッションで使いました。

Presenter Notes(□み上げない**):**

- 本番前に確認**：** Fitbit の機種は Fitbit 自身の 2020 年の NuttX の講演から。Xiaomi の数字は報告値。

8 — Sony と NuttX の歴史 (4:01 → 4:50)

画面**：** 見出し「Sony's history with NuttX」。左に縦のタイムライン — 2013–15 Early adoption(LC823425 / LC823450 に移植、最初の商用の音声製品は2015年)/ 2016–17 Commercial products(Walkman NW-WS620 ˊ SMR-10 ˊ ICD-TX800 ˊ ICD-PX470)/ 2018 Xperia Ear Duo & SPRESENSE / 2019–20 Upstream collaboration / 2020s Edge sensing。右に製品の写真3枚(Walkman NW-WS620 ˊ Xperia Ear Duo ˊ SPRESENSE)。最下段：「 Sony's NuttX journey began with audio products, expanded through SPRESENSE, and evolved into long-term open-source collaboration.」出典はスライド下の行にリンクで置く。

話す **— Essential(**必ず話す**):**

- Sony は10年以上、NuttX を使っています。公開されている記録は2013年から。Sony のエンジニアが、音声プロセッサに NuttX を移植し始めました。最初の商用の音声製品は2015年に出荷されました。
- 2018年には、Xperia Ear Duo が Sony のチップと NuttX で出荷されました。SPRESENSE も登場しました。GNSS、音声、カメラに対応した開発ボードです。
- その後、Sony のエンジニアは、プラットフォームとドライバの作業の多くを upstream に戻し、公開の NuttX の講演で共有しました。
- つまり、Sony の NuttX の歩みは、音声製品から始まり、SPRESENSE で広がり、長期のオープンソースでの協力になりました。
- 以上が Sony の歴史です。次は、私自身のことです。

話す **— Optional(**時間があれば**):**

- 公開されている製品の例には、Walkman NW-WS620、音声アンプの SMR-10、ボイスレコーダーの ICD-TX800 と ICD-PX470 があります。
- 2020年代には、カメラ、GNSS、接続性、エッジセンシングでプラットフォームの能力が伸び続けています。これは技術的な能力の話で、すべての機能が特定の製品に載ったという主張ではありません。

Presenter Notes(□み上げない):

- 参考資料(読み上げない)。 M. Ishikawa and K. Okamoto, SMP and Networking support on NuttX / LC823450, 2018: https://events.nuttx.apache.org/wp-content/uploads/2019/11/MIshikawa_nx2019.pdf 。 Sony, Running NuttX on Spresense, NuttX Workshop 2019: https://events.nuttx.apache.org/wp-content/uploads/2019/11/TKoizumi_nx2019.pdf 。 A. Jerpelea, A Journey from Fork to Mainline, NuttX Online Workshop 2020: https://events.nuttx.apache.org/index.php/past-events/nuttx-online-workshop-2020/ 。 Apache NuttX, Sony CXD56xx documentation: https://nuttx.apache.org/docs/latest/platforms/arm/cxd56xx/index.html 。 製品画像: Sony NW-WS620 https://www.sony.jp/products/picture/NW-WS620.jpg 、Xperia Ear Duo https://www.sony.jp/xperia/product/smartproducts/xea20/img/main01.jpg 、SPRESENSE https://developer.spresense.sony-semicon.com/

9 — Experience: SPRESENSE と AITRIOS での私の経験 (4:50 → 6:01)

画面: 見出し「My Experience with NuttX」。写真つきのカード2枚。 SPRESENSE — Satellite Project(人工衛星 ARICA の写真)/ AITRIOS — Edge AI Sensing Platform (edge AI カメラの写真)。最下段:「 I wanted a secure way to access and troubleshoot remote embedded devices.」

話す **— Essential(必ず話す):**

- 次は、NuttX との私自身の経験です。2つの話があります。
- 1つ目は SPRESENSE。Sony の低消費電力のマイコンボードで、NuttX が動きます。青山学院大学の ARICA-2 という CubeSat が、SPRESENSE を搭載コンピュータに使っています。ガンマ線バーストの速報を送る衛星で、2026年4月に打ち上げられました。私は社内のボトムアップの技術活動として、このプロジェクトに参加しました。
- 2つ目は AITRIOS、Sony の edge AI センシングプラットフォームです。edge AI カメラの管理や、エッジでの推論に使われます。私は edge AI カメラのシステムの導入と運用に関わりました。私が関わったシステムでは、ボードは ESP32 で、Apache NuttX が動いていました。
- 軌道上の衛星も、現場に置かれたカメラも、私の机からは遠い場所にあります。この経験から、遠くの組み込み機器に安全にアクセスして、困ったときに調べられる手段がほしいと思うようになりました。それが、次に話す課題です。

話す **— Optional(時間があれば):**

- AITRIOS のカメラがすべてこの構成というわけではありません。

Presenter Notes(□み上げない):

- 本番前に確認: ARICA の写真は第三者の画像。プロジェクトの許諾を取るか、差し替える。

10 — 目次 ②The Problem & Solution (6:01 → 6:11)

画面: 目次と同じリスト。②The Problem & Solution だけハイライト、他は薄い。

話す(一言): では、課題と解決です。

間: ハイライトされた行を指して一拍。

11 — Problem: 遠隔の機器へ、どう安全にアクセスするか (6:11 → 6:59)

画面: 見出し「Problem: How Can We Securely Access Remote Devices?」。 Engineer → Internet → Router/network → Wi-Fi → NuttX device の図(変更なし)。経路全体に必要な安全な道と、WPA2/WPA3 が守る範囲(Wi-Fi の一区間のみ)の対比。下に3枚のカード: Devices are far away / Remote diagnosis / A secure path。最下段:「We needed a secure way to access and troubleshoot remote NuttX devices.」

話す — **Essential(必ず話す):**

- 私が解きたかった課題です。
- 組み込み機器は、遠くに設置されることがよくあります: 顧客の現場、工場、別のネットワーク。何か起きたとき、技術者はログインして機器を調べ、直したい。でも、現地まで行きたくはありません。
- Wi-Fi のセキュリティは、アクセスポイントまでの無線リンクを守ります。大事ですが、守るのは一区間だけです。インターネットを越えて機器に届く方法を与えてくれるわけではありません。そのためには、別の安全な経路が要ります。
- そこで問いは、遠隔の機器にどう安全にアクセスするか、です。

話す — **Optional(時間があれば):**

- ここでの Wi-Fi セキュリティは WPA2 や WPA3 のことです。同じ必要は、edge AI や産業用 IoT など、遠隔のシステム全般にあります。

12 — Options: 遠隔の機器へ届く方法 (6:59 → 7:49)

画面: 見出し「Options: Ways to Reach a Remote Device」。 6つの手段(ポートを開ける / クラウド経由の HTTPS・MQTT / リバーストンネル・リレー / ルーターの VPN / キャリアの閉域網 / 機器の VPN)を、任意の IP サービスに届くか・間にサーバーが要らないか・どこまで守られるか、で比べた表。最後の行(機器の VPN)が今回の選択。 最下段:「Every option has trade-offs. I chose a VPN on the device, so the device itself is the end of the tunnel.」

話す — **Essential(必ず話す):**

- 方法はいくつかあります: ポートを開ける、クラウド API や MQTT ブローカーを使う、リバーストンネルを張る、ルーターの VPN を使う。
- どれにも得失があります。HTTPS と MQTT はデータには優れていますが、機器上のシェルや Web ページは得られません。ルーターの VPN はうまく働きますが、守りはルーターで終わります。
- どれも安全にできます。守る範囲が違うだけです。
- このプロジェクトでは、機器自身に標準の VPN を載せたいと考えました。そうすれば、機器がトンネルの終点になります。

Presenter Notes(□み上げない):

- **Tailscale** や **ZeroTier** について聞かれたら: これらも機器上の VPN だが、今のところマイコンより大きな OS が要ることが多い(スライドの脚注)。

13 — Solution: 機器に VPN を載せる (7:49 → 8:49)

画面: 見出し「Solution: A VPN on the Device」。 Engineer/Linux/Windows ⟷ WireGuard トンネル ⟷ NuttX device の図(変更なし)。その下に小見出し「Why WireGuard?」と3枚のカード: Small & simple(コード行数の小さな棒グラフ:

IPsec 約42万 / OpenVPN 約12万 / WireGuard 約3,800、2018年)/ Modern cryptography(ChaCha20-Poly1305、Curve25519)/ UDP-based tunnel。最下段:「Run a lightweight, standard VPN directly on the NuttX device.」

話す **— Essential(**必ず話す**):**

- では私の解決方法は、機器自身に VPN を載せることです。なぜ WireGuard か。
- まず、小さくてシンプルです。2018年の講演で、作者がコード行数を比べました: WireGuard は約4,000行、OpenVPN は約12万行、IPsec は40万行超。特定の実装の古い数字ですが、差は大きく、マイクロコントローラでは意味があります。
- 次に、モダンな暗号を使います。そして、素の UDP の上のトンネルです。Linux や Windows など既存の WireGuard ピアと話せます。マイクロコントローラ自身が、トンネルの終点になります。
- 考え方は単純です: 軽量で標準の VPN を、NuttX の機器の上で直接動かす。NuttX のどこに入るかを見ましょう。

話す **— Optional(**時間があれば**):**

- 暗号は ChaCha20-Poly1305 と Curve25519 です。詳細には触れません。

Presenter Notes(□み上げない**):**

- 行数は WireGuard 作者の Linux Plumbers 2018 の講演(5枚目、https://www.wireguard.com/talks/lpc2018-wireguard-slides.pdf)から。古く、特定の実装の数字で、同じ講演の4枚目は OpenVPN の数字が少し違う。

14 — Architecture: NuttX ネットワークスタック上の WireGuard (8:49 → 9:25)

画面**:** Application → TCP/UDP/IP → wg0 (NET_LL_TUN netdev) → WireGuard encrypt/decrypt → UDP socket → wlan0/usrsock・物理ネットワーク、の層状図。左右に TX/RX 両方向の矢印。

話す **— Essential(**必ず話す**):**

- wg0 が入る位置です。アプリケーションがデータを送ると、TCP・UDP・IP を通って降りてきて、他のネットワークデバイスと同じように wg0 に届きます。WireGuard が暗号化し、UDP パケットとして出ていきます。受信は同じ道を逆向きにたどります。
- ここから先の私の仕事は、この真ん中の部分を、上下の層を変えずに作ることです。
- では、どうやって NuttX に持ち込んだか。それが開発計画です。

話す **— Optional(**時間があれば**):**

- ボード上では、パケットは wlan0 を通って出ます。Wi-Fi チップがソケット経由で動く場合は usrsock を通ります。

15 — 目次 ③The Development Plan (9:25 → 9:35)

画面**:** 目次と同じリスト。③The Development Plan だけハイライト、他は薄い。

話す**(**一言**):** 次は開発計画です。既存の WireGuard 実装を、どう再利用できるでしょうか。

間**:** ハイライトされた行を指して一拍。

16 — Porting Strategy: wireguard-lwip から始める (9:35 → 10:31)

画面: 見出し「Porting Strategy: Start from wireguard-lwip」。一番上に GitHub のリンク (github.com/smartalock/wireguard-lwip、クリックできる)と「An existing open-source WireGuard for lwIP」。その下に大きな3段: KEEP — Protocol & crypto(緑、小さな修正2件つき)/ ADAPT — Platform hooks(金)/ REPLACE — lwIP network glue(赤)。最下段に一文。

話す — **Essential(必ず話す):**

- WireGuard をゼロから書いたわけではありません。
- wireguard-lwip から始めました。lwIP ネットワークスタック向けの、オープンソースの WireGuard 実装です。リンクはスライドにあります。プロトコルも暗号もすでにあるので、移植に集中できました。
- コードを3つに分けました。1つ目は KEEP: プロトコルと暗号。再利用しました。暗号はそのままで、プロトコル□体には小さな修正が2つ要りました。2つ目は ADAPT: プラットフォームフック。時計や乱数など、OS に依存する少数の関数です。3つ目は REPLACE: lwIP の接着部分。NuttX には lwIP がないので、この層が□当の移植作業です。
- では、lwIP は NuttX にどう対応するのでしょうか。

話す — **Optional(時間があれば):**

- 4つのフックは、単調な時計、乱数、TAI64N 形式の現在時刻、システムが高負荷かの確認です。2つのプロトコルの修正は、レビューで見つけました。

Presenter Notes(□み上げない):

- 本番前に確認: 4つのプラットフォームフックは、単調な時計・乱数・TAI64N の時刻・高負荷の確認(`wireguard-platform.h`)。スライドではなくノートから言う。

17 — Mapping lwIP Networking to NuttX (10:31 → 11:10)

画面: 見出し「Mapping lwIP Networking to NuttX」。 lwIP と NuttX の対応表 — ネットワークデバイス(`struct netif` → `struct net_driver_s`)/ パケットバッファ(`pbuf` → `iob`)/ 登録(`netif_add()` → `netdev_register()`)/ 送信(output コールバック → `d_txavail` → `devif_poll()`)/ UDP の「ワイヤ」(`udp_pcb` → UDP ソケット)。注記「考え方の対応で、API の1対1の置換ではない」。最下段:「 Porting meant using NuttX's own building blocks, not bringing lwIP into NuttX.」

話す — **Essential(必ず話す):**

- これが移植で一番大事な判断でした。lwIP を NuttX に持ち込まず、NuttX 自身の部品を使いました。
- lwIP のネットワークインターフェースは、NuttX の net_driver_s になります。lwIP のパケットバッファは、NuttX の I/O バッファに近いものです。インターフェースの追加はデバイスの登録になり、 wg0 はトンネル型のデバイスとして登録します。wg0 の下のワイヤは、素の UDP ソケットです。
- これらの API はまったく同じではありませんが、似た役割を持っています。この設計で、最初のバージョンを作れました。

話す — **Optional(時間があれば):**

- lwIP ではインターフェースが netif、バッファが pbuf です。NuttX ではバッファが iob です。送信では、lwIP は output コールバックを呼びます。NuttX ではドライバの txavail 関数が呼ばれ、ドライバが devif_poll でスタックに問い合わせます。

18 — 目次 ④Version 1: Application-Based Implementation (11:10 → 11:20)

画面: 目次と同じリスト。④Version 1: Application-Based Implementation だけハイライト、他は薄い。

話す(一言): では、Version 1、最初に動いたトンネルです。

間: ハイライトされた行を指して一拍。

19 — Version 1: NuttX のアプリケーションとしての WireGuard (11:20 → 12:08)

画面: 見出し「Version 1: WireGuard as a NuttX application」。「GOAL」の枠(Run a standard WireGuard tunnel as a NuttX application. / Reuse the existing core and implement the NuttX integration.)。その下に3列の表(Layer / Work needed / In this port): Protocol & crypto — Almost none、暗号はバイト単位で同一、プロトコル□体は 小さな修正2つ(約3,100行を再利用・約40行を変更)/ OS hooks — Four functions(約120行を新規)/ Network glue — The real work、netdev + UDP socket + RX task(約1,200行を新規)。最下段は2行の Takeaway: Version 1 proved that WireGuard could run on NuttX. / It also exposed OS and networking problems that the first tunnel did not reveal.

話す — **Essential(必ず話す):**

- Version 1 は、NuttX のアプリケーションとしての WireGuard で、FLAT ビルドです。目標は、ファームウェアを作り直さずに、実行時に設定できる標準のトンネルでした。
- この表は、各層の作業です。プロトコルと暗号は wireguard-lwip から来て、小さな修正は2つだけでした。OS フックは小さいです。□当の作業はネットワークのつなぎ込み: netdev、UDP ソケット、受信タスクでした。新規のコードは約1,200行でした。
- 動きました。でも、最初の ping の成功では見えなかった問題も出てきました。どこが難しかったかを見ましょう。

話す — **Optional(時間があれば):**

- wireguard-lwip から約3,000行が来ました。OS フックは約120行です。
- 2つの小さな修正のうち1つは、wireguard-lwip 自身のバグでした: レート制限が一度も効いていませんでした。

Presenter Notes(□み上げない):

- 行数は git の履歴から数えた物理行。再利用: 取り込んだ wireguard-lwip のファイル(約3,100行)。修正: 約40行の変更(2026-09-18)。OS フック: nuttx-platform.c、最初に動いたトンネル(2026-08-13)で118行。つなぎ込み: nuttx-wireguardif.c と .h、その時点で約1,200行。今のファイルは実行時設定などで約2倍の長さになっている。

20 — Where it was hard: 6つのバグ (12:08 → 12:43)

画面: 見出し「Where it was hard: six bugs」。6つのバグの一覧表 (バグ / 出た場所 / 教訓): 1 `wg` コマンドが終わると受信スレッドが止まる / 2 ping は通るのに TCP が死ぬ (in detail) / 3 「実害なし」が10倍(in detail) / 4 スタック94%、設定保存でボードが固まる、実機だけ / 5 NuttX 自身の ChaCha20-Poly1305(part 5) / 6 3KB のカーネルスタックに 6KB の配列(part 5)。GitHub への案内と QR コードは置かない。Takeaway: 多くは通る試験の陰に隠れていた。

話す — **Essential(必ず話す):**

- 最初のトンネルはすぐに動きました。時間の大半は6つのバグに使いました：アプリケーション版で4つ、カーネル版でさらに2つ。
- 多くは、ハンドシェイクや ping のような通る試験の陰に隠れていました。だから最初は、違う場所を探していました。
- このうち2つを詳しく話します。教訓が誰にとっても役に立つからです。バグ 5と6は、あとのカーネルの部で出てきます。

話す **— Optional(**時間があれば**):**

- バグ1: 受信スレッドは、それを起動した wg コマンドが終わると止まりました。detach が決めるのはメモリがいつ解放されるかだけで、スレッドの寿命ではありません。だからカーネルドライバは、スレッドを自分で持っています。
- バグ4: 実機だけが見せてくれました。ボード上で、受信タスクのスタックは94%使われていました。シミュレータはスタックが大きいので、見えませんでした。実機で測りましょう。

Presenter Notes(□み上げない**):**

- 6つすべてのログと修正は GitHub のリポジトリにある。
- バグ**1**と**4**には、もう専用のスライドがない。話は上のオプションにある。詳しい版(バグ1の `SO_RCVTIMEO` の回り道を含む)は Q&A 資料にある。

21 — バグ②: ping は通るのに TCP が死ぬ (12:43 → 13:29) *( )*

画面**:** 見出し「Bug ②: Ping Works. TCP Dies.」(オレンジ〜赤)。左の枠 SYMPTOM: ping は 0% loss / TCP のハンドシェイクは完了し、その後で接続が死ぬ / `EBADF` — “Bad file descriptor”。右の枠 ROOT CAUSE & FIX: `wg_rx` タスクからの送信は成功、TCP のデータを送るシステムのワークキューからは失敗 / 原因: ファイルディスクリプタは1つのタスクグループに属し、別のタスクグループでは有効でない / 修正: `struct socket` に対する `psock_*()` を使う(fd なし、どのタスクからも使える)。Takeaway は「If only some tasks fail, look at what each task owns. Here, it was the file descriptor.」

話す **— Essential(**必ず話す**):**

- このプロジェクトで一番好きなバグです。
- 同じトンネルの上で、ping は完璧に通り、TCP のハンドシェイクすら完了しました。でも、その後 TCP 接続が死にました。ログには E-B-A-D-F、不正なファイルディスクリプタが出ていました。
- 送信は受信タスクからなら成功し、別のタスクグループ、システムのワークキューからは失敗しました。
- NuttX では、ファイルディスクリプタは、それを開いたタスクグループに属します。壊れていたのではなく、別のタスクグループからは見えなかったのです。
- 修正: NuttX の内部ソケット API を、struct socket に対して使いました。ただのメモリなので、どのタスクからでも使えます。

話す **— Optional(**時間があれば**):**

- カーネル版がソケットを struct socket として持っているのも、この理由です。

22 — バグ③: 「実害なし」が実は10倍遅かった (13:29 → 14:14) *(3)*

画面**:** 見出し「Bug ③: “No Real Impact” Was 10× Slower」(オレンジ〜赤)。2つの枠を並べる: QUICK FIX — Check every 50 ms(測定: 約10倍遅い)/ REAL FIX — Event-driven wake-up (`psock_poll()` のコールバックとセマフォ)。Takeaway: 測らない応急処置はタダに見える。測ること。

話す **— Essential(**必ず話す**):**

- 次のバグは、測っていなかったことの話です。
- ファイルディスクリプタを使わない修正は、poll も壊しました。そこで応急処置として、50ミリ秒ごとにソケットを確認しました。コストは CPU を起こすことだけだと思っていました。間違いでした。届いたパケットはすべて最大50ミリ秒待たされ、約10倍遅くなりました。
- □当の修正は、正しい待ち方です: ソケットがコールバックし、データが届いたらセマフォがスレッドを起こします。
- 教訓: 測らない応急処置は、タダに見えることがあります。だから、測りましょう。

話す **— Optional(**時間があれば**):**

- poll が壊れたのは、poll がファイルディスクリプタで動くからです。

23 — Version 1 のデモ: 最初の動画 (14:14 → 14:38) *(2)*

画面: 左に動画(静止画 + 再生ボタン。クリックするとスライドの中で YouTube の動画が再生され、スライドを離れると止まる。公式 Windows WireGuard クライアントと、ESP32-S3 上の NuttX への telnet セッションが並んだ画面)。右に手順 — telnet 10.10.0.2 / uname -a ˊ ifconfig / webserver & / ブラウザで http://10.10.0.2/ — と条件(ESP32-S3 DevKit ˊ apps 版(FLAT) ˊ USB は抜いてある、相手は公式 Windows クライアント)、小さく YouTube へのリンク。Takeaway: Version 1 was enough for a demo. Next, I made it a real NuttX driver.

話す **— Essential(**必ず話す**):**

- Version 1 の動画の1コマです。トンネル越しに telnet でログインし、ボード上の小さな Web ページを開きました。相手は公式の Windows 版 WireGuard です。
- Version 1 でもデモには十分でした。次に、これを□物の NuttX のドライバにしました。

Presenter Notes(□み上げない**):**

- 動画の再生は必須ではない: Essential の話は静止画のままで成り立つ。再生するにはサムネイルをクリック。インターネットと、Web ページ(GitHub Pages かローカルサーバー)が要る。ファイルとして開いた場合は、クリックで YouTube が新しいタブで開く。スライドを離れると動画は止まる。
- これは **apps** 版。この後のカーネル版の結果と混ぜて話さない。

24 — 目次 ⑤Version 2: The Kernel Driver (14:38 → 14:48)

画面: 目次と同じリスト。⑤Version 2: The Kernel Driver だけハイライト、他は薄い。

話す**(**一言**):** では、Version 2、カーネルドライバです。

間: ハイライトされた行を指して一拍。

25 — Version 2: NuttX のカーネルドライバとしての WireGuard (14:48 → 15:44)

画面: 見出し「Version 2: WireGuard as a NuttX kernel driver」。左右2枚の枠 — 左 BEFORE — VERSION 1:「 An application」(`apps/` にあり、カーネル内部(`psock_*`)に手を入れる。FLAT ビルドでしか動かず、in-tree のドライバとしては合わない)/ 右 AFTER — VERSION 2:「 A kernel driver」(`drivers/net/wireguard/` に `wg0`、小さな `wg` コマンドが ioctl で設定。FLAT · PROTECTED · KERNEL のために作った)。その下に BUILD MODES の3行: FLAT — アプリとカーネルが1つのアドレス空間 / PROTECTED — MPU がアプリとカーネルの メモリを分ける / KERNEL — MMU が別々のア

ドレス空間を与える。最下段:「The goal: a proper kernel driver for every NuttX build mode, in a form that is easier to review upstream.」

話す **— Essential(**必ず話す**):**

- なぜカーネルに移したのか。
- Version 1 はアプリケーションです。カーネルの内部に手を入れています。アプリとカーネルが1つのアドレス空間を共有する FLAT ビルドなら、それで動きます。でも in-tree のドライバとしては合わず、両者が分けられる PROTECTED や KERNEL ビルドでは動きません。
- そこでデバイスを、他のネットワークドライバの隣のカーネルに移し、小さな wg コマンドが ioctl で設定する形にしました。
- 目標は、すべてのビルドモードで使える□物のカーネルドライバで、NuttX の構造と upstream のレビューにより合う形にすることです。受け入れられるという意味ではありません。NuttX の作りに沿う、という意味です。

話す **— Optional(**時間があれば**):**

- PROTECTED ビルドでは MPU がメモリを分けます。KERNEL ビルドでは MMU がそれぞれにアドレス空間を与えます。

26 — カーネル側の設計: 全体像 (15:44 → 16:22)

画面: 見出し「The Kernel Design: The Big Picture」。図は2つの帯 — 上が USER SPACE(青みがかった地、`wg` コマンドだけ。apps/system/wg ´ genkey · set · show)、下が NUTTX KERNEL(緑がかった地)。ioctl(ポインタを含まない平らな構造体1つ。秘密鍵は書き込み専用)がユーザー/カーネルの境界を越えて、カーネル内の `wg0`(drivers/net/wireguard)に届く。`wg0` は自分の UDP ソケットと受信スレッドを持ち、タイマー・ピア・クッキーを管理し、NuttX の暗号(crypto/)を使う。左に NuttX のネットワークスタック、右に暗号化された UDP(wlan0 / usrsock → ピア)。

話す **— Essential(**必ず話す**):**

- ドライバの形です。上がユーザー空間、下が NuttX カーネルです。
- ユーザー空間にあるのは、小さな wg コマンドだけです。ioctl で、ポインタを含まない平らな構造体1つとして、カーネルに設定を送ります。秘密鍵は書き込み専用です。
- カーネルの中では、wg0 は普通のネットワークデバイスです。UDP ソケットと受信スレッドを自分で持っているので、Version 1 のスレッドとソケットのバグは、設計上もう起きません。

話す **— Optional(**時間があれば**):**

- 暗号は NuttX 自身のものです。プロトコル□体は今も wireguard-lwip 由来の BSD-3-Clause のコードで、そのことは LICENSE ファイルに書いてあります。

27 — Two Issues Found During Kernel Integration: カーネル統合で見つかった2つの問題 (16:22 → 17:22)

画面: 見出し「Two Issues Found During Kernel Integration」。左右2つの枠にラベル。左(青い縁、少し強調)CRYPTOGRAPHY — ChaCha20-Poly1305(NuttX `crypto/chachapoly.c`: ナンスのカウンタが RFC 8439 / WireGuard の並びと違うバイトにある。カウンタ0では一致するので、ハンドシェイクは通るが、最初より後のデータパケットが失敗する。テストベクタつきの2行の修正を、独立した変更に)/ 右(灰色の縁)KERNEL STACK — Stack Overflow(このドライバ、

KERNEL ビルドのみ: 3KB のカーネルスタックに 6KB の配列 → あふれてパニック。sim では出ない。配列はヒープへ。 ドライバ自身のバグ)。背景は両方ともニュートラル。SPRESENSE の RTC_HIRES は画面に載せず、ノートだけ。 最下段 :「 Porting WireGuard also surfaced findings worth sharing with the NuttX community.」

話す **— Essential(**必ず話す**):**

- カーネルへ移す途中で、2つの別々の問題が見つかりました。
- 1つ目は、NuttX の ChaCha20-Poly1305 のコードです。64ビットのカウンタを、WireGuard が使う RFC 8439 の並びとは違うナンスのバイトに置いています。カウンタが0のときはバイト列が同じなので、ハンドシェイクは通り、その後、最初より後のデータパケットが失敗します。修正は2行で、テストベクタをつけた独立した変更として用意しました。
- 2つ目は、私自身のドライバのバグで、NuttX のバグではありません。ピアの設定が、3キロバイトのカーネルスタックに6キロバイトの配列を置いていて、パニックを起こしました。KERNEL ビルドでだけです。配列はヒープに移しました。
- これらは小さな別々の単位に分けて用意しました。どれもまだ提出していません。

話す **— Optional(**時間があれば**):**

- 私の見る限り、現在の NuttX の master も同じナンスの並びです。
- ボード側からもう1つ。NuttX 13.0.1 と master で、高分解能 RTC のオプション CONFIG_RTC_HIRES を有効にすると SPRESENSE が起動しませんでした。原因を見つけて手元で直しました。報告は下書きまでで、まだ出していません。

Presenter Notes(□み上げない**):**

- 2026-10-13 の状況: apache/nuttx にも apache/nuttx-apps にも pull request は無い。関連する issue は閉じた GSoC の提案スレッド #18548 だけ。master との nonce の確認は 2026-10-13(crypto/chachapoly.c)。
- 言い方に注意**:** スタックの件は NuttX ではなくこのドライバの問題。提出済みと言わない。

28 — 検証**:** どこで何が動いたか **(17:22 → 17:55) ( )**

画面**:** 見出し「Testing: What Ran Where」。表(Build mode / Environment / Result): FLAT — ESP32-S3・SPRESENSE (実機)✓ Verified / KERNEL — QEMU rv-virt(Linux へのトンネル、4 CPU で負荷)✓ Verified / PROTECTED — QEMU rv-virt(1件の例外を見つけて修正、20/20)✓ Verified / KERNEL・PROTECTED — 実機 — Not tested yet。小さな注記「✓ は、ここに書いた条件で通ったという意味で、他のハードウェアへの保証ではない」。 GitHub への案内と QR コードは置かない。最下段は2行: 実機の FLAT と QEMU の KERNEL / PROTECTED で検証した / それらの実機での検証は残っている。

話す **— Essential(**必ず話す**):**

- 何をどこで検証したかを、正確に言います。
- FLAT ビルドは実機で検証しました: ESP32-S3 と SPRESENSE。KERNEL ビルドは QEMU で検証しました。PROTECTED ビルドも QEMU で検証しました。KERNEL と PROTECTED の実機での検証は、まだできていません。
- だからチェックマークは、この条件で通ったという意味です。すべてのボードへの約束ではありません。

話す **— Optional(**時間があれば**):**

- 10月12日に、upstream 向けに用意した提出シリーズそのものを ESP32-S3 で動かしました: Linux の WireGuard とハンドシェイクし、両方向の ping を、パケットの最大サイズまで通しました。
- SPRESENSE ではメモリを測りました。40回の down/up の後でも、ぴったり同じでした。

- QEMU の 4 CPU で、設定を変えながら大量に流しました。例外は出ませんでしたが、これは1回のスクリプト実行で、競合の全探索ではありません。
- PROTECTED では、繰り返すとたまに例外が出ました。原因は wg コマンドのスタックあふれで、既定値を大きくして直りました: 20回中20回。

Presenter Notes(□み上げない):

- 詳細。SPRESENSE の受信スレッドは 6KB の既定値のうち 1,472 バイトを使った。PROTECTED のスタックあふれ: このモードではシステムコールが呼び出し側のスタックで動く。CoreS3 はブートローダ経由のイメージを起動できないので、このボードでの PROTECTED はできない(WireGuard の問題ではない)。記録は docs/upstream/evidence にある。

29 — 目次 ⑥Live Demo (17:55 → 18:05)

画面: 目次と同じリスト。⑥Live Demo だけハイライト、他は薄い。

話す(一言): ライブデモの時間です。

間: ハイライトされた行を指して一拍。

30 — ライブデモ: Stack-chan を WireGuard 越しに (18:05 → 21:05)

画面: 見出し「Live Demo: Stack-chan over WireGuard」。 1枚だけで、デモ中はこの1枚を出したまま、端末を横に並べる。 上半分 NETWORK SETUP: Laptop(WireGuard peer、VPN IP 10.10.0.1)と Stack-chan(Apache NuttX + WireGuard、 VPN IP 10.10.0.2)が、Smartphone(Wi-Fi hotspot、ローカルネットワークを提供)に点線の Wi-Fi でつながり、 ラップトップと Stack-chan の間に太い赤の「Encrypted WireGuard Tunnel」が1□。スマホは WireGuard のピアとして描かない。 下半分 LIVE DEMO STEPS: 1 VPN Connection(`wg show`)/ 2 Remote Login(`telnet 10.10.0.2`)/ 3 Network Interface(`ifconfig` → `wg0`)/ 4 Control Stack-chan(表情を変える・声を出す)。 隅に小さな Stack-chan の写真(GitHub へのリンク)。Version の表記、Takeaway の枠、動画へのリンクは置かない。

話す — **Essential(必ず話す):**

- このスライドは出したままにして、端末をその横に置きます。
- ラップトップと Stack-chan は、どちらも私のスマホのテザリングにつないでいます。スマホはローカルネットワークを提供するだけです。ラップトップと Stack-chan の間には、WireGuard のトンネルが1□あります。ラップトップは 10.10.0.1、Stack-chan は 10.10.0.2 です。スマホは WireGuard のピアではありません。暗号化されたパケットを運ぶだけです。
- ステップ1、VPN の接続。ラップトップで wg show を打ちます。ハンドシェイクが済んでいて、トンネルが立っています。
- ステップ2、リモートログイン。telnet で、トンネル越しに Stack-chan につなぎます。これで NuttX のシェルの中です。
- ステップ3、ネットワークインターフェース。ifconfig を打ちます。スマホのネットワーク上の wlan0 と、VPN のアドレスを持つ wg0 が見えます。
- ステップ4、操作。まず表情を変えます。次に、しゃべらせます。
- つまり Stack-chan は NuttX の機器で、私は WireGuard 越しに遠隔操作しています。ひとこと: このロボットはアプリケーション版の FLAT ビルドで動いています。カーネルドライバは、同じチップで別のテストで動かしました。
- デモは以上です。では、まとめます。

話す — **Optional(時間があれば):**

- 時間があれば、パケットを見せます。トンネルの外、スマホのネットワークでは、暗号化された UDP だけが見えます。トンネルの中では、telnet の文字列が見えます。そのあと、もう一度 wg show で、転送バイト数が増えています。
- 音声は事前にラップトップで作ったもので、クラウドの API は使っていません。

Presenter Notes(□み上げない):

- 操作(順番どおり)。1) scripts/stackchan/demo.sh show(wg show): ピア、最新のハンドシェイク、スマホのネットワーク上のエンドポイント、転送量のカウンタが見えるはず。 2) demo.sh login(telnet 10.10.0.2): NuttX のシェルのプロンプトが見えるはず。 3) セッション内で ifconfig: wlan0 と、10.10.0.2 を持つ wg0 が見えるはず。 4) stackchan face happy(ほかに neutral・sad・angry・sleepy・surprised)、次に stackchan say http://10.10.0.1:8000/hello.wav: 音声ファイルは同じトンネルを通って届き、表情が変わって口が動く。任意: demo.sh outside(eth0 の tcpdump、UDP 51820)と demo.sh inside(wg0 の tcpdump、TCP 23)、そのあともう一度 show。各ステップの後、ボードの反応を待つ。
- どのバージョンが動いているか: ロボットのファームウェアはアプリケーション版(Version 1、FLAT)で、Dockerfile の esp32s3-stackchan ステージで作る。カーネルドライバ(Version 2)は、2026-10-12 に同じ ESP32-S3 で別のビルドで動かしたが、Stack-chan のアプリは入っていない。デモを Version 2 だと言わない。
- 本番前: スマホのテザリングを ON(2.4 GHz / 互換性優先)、ラップトップを接続、Docker Desktop を起動。Stack-chan を `nsh_wifi.py` でつなぐ(パスフレーズはボードに保存しない)、`wg_setup.py` を1回実行。`stackchan say` を1回試す。テザリングを入れ直すとアドレス帯が変わることがあるので、そのときは `nsh_wifi.py` と `wg_setup.py` をやり直す。`say` の途中でボードが止まることがまれにある(約20回に1回)。止まったら電源を入れ直して同じ手順。
- 画面に出さない: `wg showconf`(秘密鍵を表示する)、`%USERPROFILE%\stackchan-wg` の鍵ファイル、テザリングのパスフレーズ。
- フォールバック: ライブが止まったら、そう言って、次のスライド(バックアップの動画。サムネイルをクリック)を開くか、まとめへ進む。

31 — ライブデモ: 予備の動画 (21:05 → 21:05)

画面: 見出し「Live Demo: backup video」。左に動画のサムネイル(クリックでスライドの中で YouTube の動画が再生。スライドを離れると止まる)。右に小さな枠「THREE TERMINALS」: 左 control / 中央 tcpdump, encrypted / 右 tcpdump, plaintext、動画へのリンク。ライブデモが止まったときの予備。使わなければ飛ばす。

話す **— Essential(必ず話す):**

- これは、同じデモの録画です。

Presenter Notes(□み上げない):

- 予備のみ: ライブデモがうまくいったら飛ばす。時間には含めない。左の端末: 操作(wg show、telnet、ifconfig、表情、say)。中央: トンネルの外の tcpdump、暗号化された UDP だけ。右: トンネルの中の tcpdump、平文。再生はサムネイルをクリック。インターネットと、Web ページ(GitHub Pages かローカルサーバ)が必要。スライドを離れると止まる。

32 — 目次 ⑦Summary & Reflections (21:05 → 21:15)

画面: 目次と同じリスト。⑦Summary & Reflections だけハイライト、他は薄い。

話す(一言): 最後に、まとめと振り返りです。

間: ハイライトされた行を指して一拍。

33 — Summary (21:15 → 21:57)

画面: 見出し「Summary」。3つの行で、それぞれ短い見出しと1文: ① Secure Communication(WireGuard を Apache NuttX に持ち込み、標準の WireGuard ピアと安全な IP 通信ができるようにした)/ ② Application to Kernel Driver(アプリケーションからカーネルの ネットワークドライバへ発展させ、実機と QEMU で検証した)/ ③ Toward Upstream Contribution(ドライバと関連する修正を、 Apache NuttX コミュニティのレビューに向けて準備している。小さく「小さな別々の変更として用意した。まだ何も提出していない」)。 `wg0` などの内部用語や、PR の分け方・ファイルパスは載せない。

話す — **Essential(必ず話す):**

- まとめます。私たちは WireGuard を Apache NuttX に持ち込みました。NuttX の機器が、標準の WireGuard ピアと、暗号化された IP トンネルで話せるようになりました。
- アプリケーションからカーネルのネットワークドライバへ発展させました。実機で、そして QEMU のカーネル系のビルドモードで検証しました。
- 次に、ドライバと関連する修正を、NuttX コミュニティのレビューに向けて準備しています。小さな別々の単位で、まだ何も提出していません。
- ここまでが技術の話です。次は、私が学んだことを話します。

Presenter Notes(□み上げない):

- 「マージされたか」と聞かれたら: まだ。何も提出していない。提出の候補であって、マージできるという主張ではない。
- 提出の状況**(2026-10-13):** apache/nuttx にも apache/nuttx-apps にも、この作業の pull request は無い。関連する issue は閉じた GSoC の提案スレッド(#18548)だけ。ブランチは発表者のフォークにあるだけ。「提出した」と言わない。

34 — What I Learned: 学んだこと (21:57 → 23:21)

画面: 見出し「What I learned」。カード3枚: 1 アプリのユーザーから OS のコントリビューターへ / 2 AI はコードを書くのを助けてくれたが、理解するのは私の仕事だった(強調)/ 3 歓迎してくれるオープンソースコミュニティ (良いドキュメントと貢献ガイド、 活発な議論とレビュー、人が責任を持つ AI 支援開発への前向きさ、 CI・ハッカソン・イベントが新しい人の参加を助ける)。 最下段:「 I hope to keep contributing to Apache NuttX and the open-source community, and I look forward to seeing NuttX continue to grow.」

話す — **Essential(必ず話す):**

- 最後に、学んだことを3つ話します。
- 1つ目。私は NuttX にアプリケーション開発者として来ました。今回は OS の中で、ドライバとカーネルを扱いました。
- 2つ目。これははっきり言いたいです。このプロジェクトでは AI、Claude をたくさん使いました。調査もコードも、ずっと速くなりました。でも組み込み OS の内部は私には新しく、AI が書いたコードをそのまま受け取ることはできませんでした。質問し、NuttX のソースを読み、動きを試し、自分で説明できるようになるまで続けました。AI はコードを書くのを助けてくれましたが、理解するのは私の仕事でした。
- 3つ目、コミュニティです。NuttX には良いドキュメントと貢献ガイドがあり、議論とレビューが活発です。AI 支援の開発にも前向きですが、責任は人が持ちます。CI やハッカソンも、新しい人の参加を助けています。オープンソースは、コードが公開されて

□□

- NuttX □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□NuttX □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

□□ — Optional(□□□□□□□):

- NuttX □□□AI □□□□□□□□□□□□ Assisted-by □□□□AI □ Signed-off-by □□□□□□□

Presenter Notes(□□□□□□□):

- AI □□□□□□□□□□□□□□□□□□□: □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

35 — Acknowledgements: □□ (23:21 → 24:07)

□□**:** □□□□Acknowledgements□□□□□□□□□□□□□□□□□□□□□□□□□□□□□(□□□□□□□□)□□□1□: Alin Jerpelea(□□□□□ Community Over Code □□□□□□□□□□□□□)/ The Apache NuttX Community (□□□□□□□□□□□□□□□)/ Sony's NuttX Developers, Past and Present(NuttX □□□□□□□□□□□□□□□)/ The Apache Software Foundation & Community Over Code Organizers(□□□□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□)□□□□ □□ → □□□□□□ → □□□□□□□

□□ — Essential(□□□□□):

- □□□□□□□□□□□□□□□
- □□□Alin Jerpelea □□□□□□□□ Community Over Code □□□□□□□□□□□□□□□
- □□□Apache NuttX □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□GSoC □□
- 3□□□□□□□□□□ Sony □□□□□□□□□NuttX □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□Apache Software Foundation □□Community Over Code □□

36 — Thank You (24:07 → 24:20)

□□**:** □□□□Thank You□□□□□ QR □□□(□Scan to view the slides□)□□□□□□□□□□□□□□□□□2□: □□□□ (https://wwlapaki310.github.io/gsoc2026-nuttx-wireguard/docs/presentation/coc-glasgow-reveal/index.html) □ GitHub □□□□□□□□□□□□□□ QR □□□(□Slides□)□□□□□□□□□□□□□□

□□ — Essential(□□□□□):

- □□□□□□□□□□□□□
- □□□□□□□□□□□□□□QR □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ GitHub □□□□□□

Presenter Notes(□□□□□□□):

- □□□□□□**:** coc-glasgow-qa.md □□□□□□□□□□□□(□□□□□□□□#14□IOB □□□□checkpatch)□□□□□□□□□□□□□□□□□□□□□
- □□□□□□□**:** QR □□□□2□□□□□□□ https://wwlapaki310.github.io/gsoc2026-nuttx-wireguard/docs/presentation/coc-glasgow-reveal/index.html □□□□□(GitHub Pages □□□□ push □□□□□□□□□□)□

A1 — 付録: 検証: ping が見逃すもの (付録・時間に含めない)

付録: □編には含めない。Thank You の後、質問で必要になったら、最後のスライドから下矢印で開く。

画面: 見出し「Three problems that ping would never notice.」。カード3枚 — 起動のたびに同じ鍵? (SPRESENSE: 3回のリセットで3つの違う鍵。簡単な確認で、乱数の質の証明ではない)/ down の後に鍵が メモリに残る?(sim: セッション鍵は down の後すべてゼロ。静的鍵は up をやり直すために意図的に残す)/ 数時間後に出るリーク?(sim: 151回の再鍵化、100回の down/up、何も失われない。メモリは起動時の数に戻る。 1週間の試験ではない)。

話す — **Essential(必ず話す):**

- このスライドは、動くトンネルでは分からないことの話です。ping では気づかない3つの問題。
- 1つ目は鍵。NuttX は、乱数デバイスをビルド時の設定でシードできます。すると、ボードは起動のたびに同じ秘密鍵を作り、それでも全部が動きます。そこで SPRESENSE を3回リセットして、3つの違う鍵を得ました。簡単な確認で、乱数の質の証明ではありません。
- 2つ目は、シミュレータでデバイスのメモリを読みました。down の後、セッション鍵はすべてゼロでした。静的な鍵は、up をやり直せるように、意図的に残します。
- 3つ目は、これもシミュレータでの長い試験: 151回の強制的な鍵の更新と、100回の down/up。バッファは1つも失われず、メモリは起動時の数にぴったり戻りました。1週間動いたとも、シミュレータがボードと同じ動きをするとも言っていません。

A2 — 付録: 設計判断: タイムスタンプ問題には楽な答えがない (付録・時間に含めない)

付録: □編には含めない。Thank You の後、質問で必要になったら、最後のスライドから下矢印で開く。

画面: 規則(再起動の後でも、相手が見たものより新しいタイムスタンプが要る)。Chosen(リアルタイム クロック + この起動で使った最大値。Linux・wireguard-go と同じ規則 + 時計が未設定なら警告)/ Not chosen (毎回保存: flash が摩耗 / flash に範囲を予約: 確かめられなかった書き込み保証が要る / 待つ: 前回動いて いた時間だけ)。実測のペア(時計未設定 → 75秒でハンドシェイクなし、時計設定 → 4.1秒)。

話す — **Essential(必ず話す):**

- これは、試すのではなく、決めなければならなかった唯一の場所です。そして、その決定を測りました。
- WireGuard のリプレイ防止には規則があります: ハンドシェイクのたびに、再起動の後でも、相手が見たものより新しいタイムスタンプが要ります。デバイスは普通ハンドシェイクを始める側で、電池付きの時計を持っている可能性が一番低い側でもあります。
- すべてのタイムスタンプを保存すると、フラッシュがすり減ります。フラッシュに範囲を確保するのは長期的には正しい答えですが、確認できなかった書き込みの保証が要ります。そして単に待つなら、1か月動いていたデバイスは、1か月再接続できません。
- そこで、リアルタイムクロックと、この起動で使った最大値を選びました。Linux や wireguard-go も同じです。そして、時計が一度も設定されていなければ、ドライバは1回だけ警告します。
- それからボードで測りました。ボードには何も送りませんでした。相手からの ping が、ボードの代わりにハンドシェイクを始めてしまうからです。起動したままの時計では、75秒見ていてもハンドシェイクはありませんでした。時計を設定すると、4.1秒でした。だからドライバは警告を出します。